PilotFish Synchronous Response Transport – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

When a partner calls your interface and **waits for an answer**, the inbound listener holds the connection open using a **ticket**. This transport is the **hand-back**: at the end of the processing route it copies the response body and returns it to the listener that is still waiting.

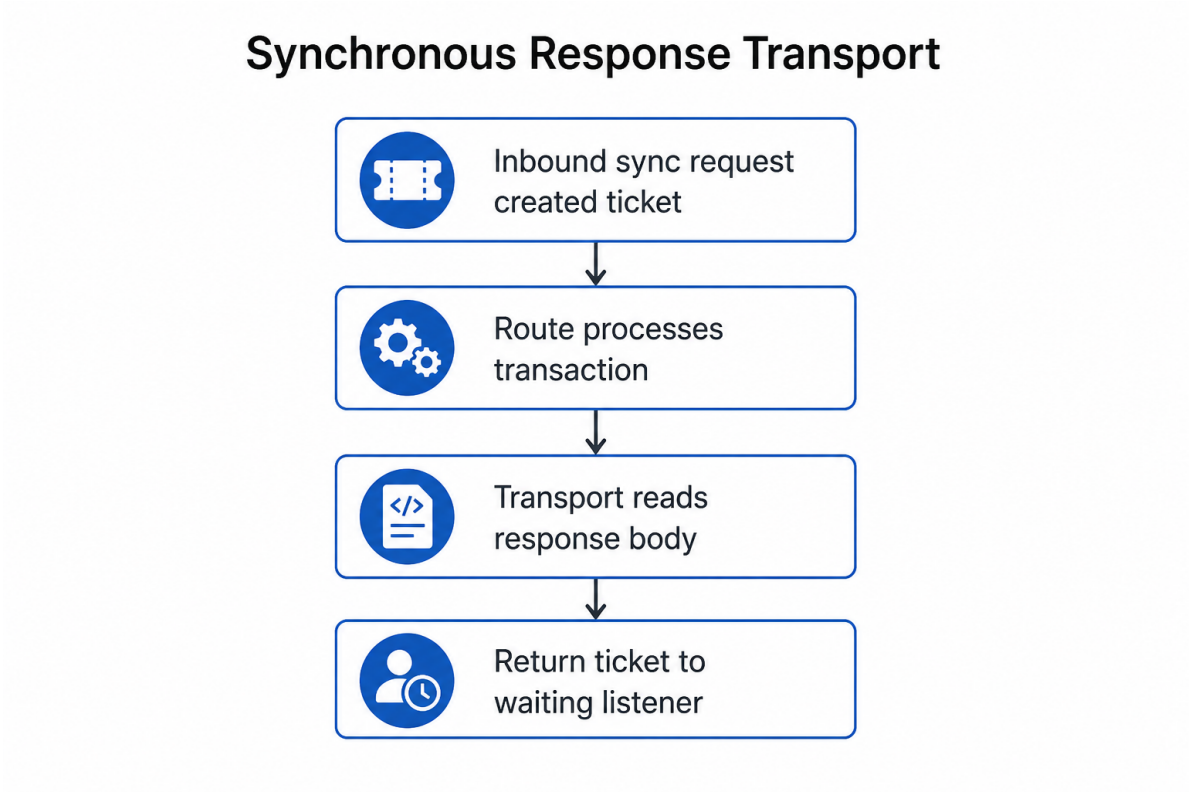

Figure 1: At a glance (plain English)

- **Finds** the synchronous response ticket placed on the transaction by the inbound listener.
- **Caches** the current message body as the response payload.
- **Returns** a cloned transaction (with that stream) to the waiting listener via `SynchronousResponseBroker`.
- **Has no configuration options** — behavior is entirely driven by the ticket attribute.

Purpose

This document is a **deep dive** into the **Synchronous Response** transport as shipped in `eip.war.hs.26R1.11`: how it validates the ticket, clones transaction data with a cached response stream, and completes the synchronous request cycle.

Provenance	Value
WAR	`eip.war.hs.26R1.11`
Module JAR	`xcs-modules-core-1.0.3.jar`
Decompiler	CFR 0.152
Implementation class	`com.pilotfish.eip.modules.internal.Synchro`
Decompiled path	`war-pipeline/decompiled/eip-26R1.11/.../Sy`
Transport type string	`Synchronous Response`
Description	`Sends a synchronous response back to the listener instance in the route that triggered this transaction.`
Help ID	`console.transport.synchronous_response`
Category	`PILOT_FISH`

> This transport defines **no module-specific configuration items**. It overrides `sendData` directly rather than `executeTransport`.

---

1. Conceptual model

Synchronous interfaces (HTTP, HL7 with sync ack, programmatic triggers, etc.) register a `SynchronousResponseTicket` on the transaction when the caller must block until a response exists. The processing route runs asynchronously through stages; this transport **delivers** the final payload back to the broker that unblocks the caller.

```
Sync listener (creates ticket on TransactionData)
        |
        v
 Route stages (transform, branch, ...)
        |
        v
 Synchronous Response Transport
   - cache body
   - read ticket attribute
   - broker.returnTicket(ticket, cloned data)
        |
        v
 Waiting listener sends response to client
```

Without a valid ticket, the transport fails fast — it cannot invent a synchronous caller.

---

2. High-level behavior (`sendData`)

From decompiled `SynchronousResponseTransport.sendData` (overrides `AbstractTransport.sendData`):

1. `checkRequiredData(data)` — standard transport precondition checks.
2. **Cache transaction body**
   - `EIPCacheManager.getInstance().getDefaultCache()`
   - `DataUtil.copyAndClose(data.getDataStream(), cache.getOutputStream())`
3. **Locate ticket**
   - Read attribute `com.pilotfish.eip.synchronousResponseTicket`
   - `null` → `EIPException("No synchronous response ticket found...")`
   - Not a `SynchronousResponseTicket` → `EIPException` with actual class name
4. **Return response**
   - `SynchronousResponseBroker.returnTicket(ticket, data.clone(cache.getInputStream()))`
5. **Mark complete** — `data.setTransactionStateComplete()` (does not call `executeTransport`).

On any exception: `data.setTransactionStateError()` then `EIPException("Error while loading transaction content", cause)`.

---

3. Configuration reference

3.1 Module-specific options

None. `SynchronousResponseTransport` does not override `getConfigurationDescriptor()`.

3.2 Inherited AbstractTransport behavior

Because `sendData` is fully overridden, the default `AbstractTransport` path (`executeTransport` + auto-complete) is **not** used. There is no empty `executeTransport` fallback.

Standard module metadata (name, route binding) still applies via `AbstractEIPModule`.

---

4. Transaction attributes

4.1 Required input

Attribute key	Type	Source
`com.pilotfish.eip.synchronousResponseTicket`	`SynchronousResponseTicket`	Stamped by synchronous-capable inbound listener / trigger

4.2 Output / side effects

- **Broker handoff** — response stream attached to cloned `TransactionData`; original transaction marked complete.

- Attributes on the clone follow whatever `TransactionData.clone(InputStream)` copies from the working transaction.

---

5. Worked example — HTTP-style sync route

1. **HTTP XML Listener** (sync mode) receives POST, creates ticket, starts route.
2. Processors transform request → response XML in transaction body.
3. **Synchronous Response Transport** (final stage):
   - Caches XML body.
   - Returns ticket with cloned data whose stream is the XML.
4. Listener writes HTTP response to client and closes connection.

If stage 3 is missing, the client hangs until listener timeout even if processing succeeded.

---

6. Known quirks and caveats

1. **No UI configuration** — cannot tune timeout or charset here; listener owns sync policy.
2. **Ticket type strict** — wrong object type fails with explicit class name in error.
3. **Stream consumed** — `copyAndClose` closes inbound stream before return.
4. **Clone semantics** — downstream code should treat broker return as terminal for this transaction's response duty.
5. **Error state** — any failure sets transaction state error before throwing.
6. **No `executeTransport`** — do not expect subclass hooks; all logic is in `sendData`.

---

7. Operational recommendations

Goal	Guidance
Route design	Place as the **last transport** (or only transport) on sync response routes
Testing	Verify ticket attribute exists in regression captures before this stage
Error routes	On failure prior to this transport, ensure listener error path releases ticket
Mixed async/sync	Do not reuse the same route terminus for async deliveries

---

8. Source index (this WAR)

Topic	Path under `war-pipeline/decompiled/eip-26R1.11/`
Synchronous Response transport	`com/pilotfish/eip/modules/internal/SynchronousResponseTransport.java`
Abstract transport	`com/pilotfish/eip/extend/AbstractTransport.java`
Broker / ticket (core)	`com/pilotfish/eip/SynchronousResponseBroker.java`, `SynchronousResponseTicket.java`
Module JAR	`war-pipeline/jars/eip-26R1.11/xcs-modules-core-1.0.3.jar`

---

9. Pipeline provenance

```
PilotFish WARs/eip.war.hs.26R1.11
  -> xcs-modules-core-1.0.3.jar
  -> CFR decompile
  -> Documents/Transports/26R1.11/PilotFish-Synchronous-Response-Transport-Reference-26R1.11.(m
```

Deep-dive documentation from WAR decompile – Synchronous Response Transport – 26R1.11 – August 2026.